ChatGPT

TypeScript/React Gradio Demos and Integrations

- **Gradio-Flow (LVivona)** – An open-source **Flask + React** web app for building AI workflows. It uses React (with React Flow) to let users drag-and-drop workflow nodes, each node embedding a Gradio or Streamlit interface. According to the README, Gradio-Flow "streams both Gradio (and later Streamlit) interfaces within a single application" [1]. In practice, the React frontend connects to Gradio apps (hosted e.g. on HuggingFace Spaces or locally) via the Gradio JS client or by embedding their UIs, allowing chaining of image/text models and tools. Key features include an interactive node-based UI, light/dark mode, and support for integrating multiple Gradio-backed tools into one React dashboard. (Repo: GitHub – LVivona/Gradio-Flow [1].)

- **DocsMaker (Abmichael01)** – A React + TypeScript documentation/AI tooling app that integrates Gradio APIs. Its frontend is built with React+Vite+TypeScript and includes UI components (e.g. an image cropper) that call Gradio endpoints via the `@gradio/client` library. For example, the `ImageCropUpload` component imports `Client` from `@gradio/client` and calls a background-removal model [2]. The project's `package.json` lists `@gradio/client` alongside React/TypeScript dependencies [3]. In use, users can crop or annotate images in the React UI and then invoke a Gradio-based model (e.g. for background removal or annotation) by calling `client.predict()`. This provides a seamless integration of Gradio-backed ML (such as image processing or annotation models) into a modern TS/React frontend. (Repo: GitHub – Abmichael01/docsMaker [2] [3].)

- **Browsernode Gradio Example** – A TypeScript example (part of the browsernode project) showing how to use Gradio's JS client in a Node/Express app. In this demo, an OpenAI-based agent runs a task and the Express server uses `Client.connect()` to call a local Gradio UI [4]. Specifically, the code initializes `Client.connect("http://localhost:7860", …)` to bind to a running Gradio app, then passes user input through the Gradio endpoint. This shows how a React/TS (or plain TS) backend can programmatically query Gradio-hosted models (e.g. an image classifier or task-oriented model) – effectively treating Gradio as an API. While this example uses an AI agent workflow (text generation via ChatGPT), the same pattern can be used for data-visualization models or other demos. (Repo: GitHub – leoning60/browsernode, see `examples/ui/gradio_demo.ts` [4].)

Each of these projects is publicly available (with links above) and demonstrates how Gradio's JavaScript/TypeScript client can be paired with React to build interactive AI demos or visualization UIs. In all cases the React app imports `@gradio/client` and invokes Gradio-hosted models (image classifiers, text generators, etc.) via the `Client.connect()` and `client.predict()` APIs [4] [2]. The result is a rich front-end (charts, image canvases, chat UIs, etc.) powered by Gradio model backends.

Sources: Official GitHub repos and docs for the above projects (see citations) [1] [3] [4] [2].

---

[1] GitHub - LVivona/Gradio-Flow: A web application with a backend in Flask and frontend in React, and React flow node base environment to stream both Gradio (and later Streamlit) interfaces, within a single application.
https://github.com/LVivona/Gradio-Flow

2 ImageCropUpload.tsx

https://github.com/Abmichael01/docsMaker/blob/a3a970daa73e218a8d362653adf8438a2480f0b1/src/components/ui/ImageCropUpload.tsx

3 package.json

https://github.com/Abmichael01/docsMaker/blob/a3a970daa73e218a8d362653adf8438a2480f0b1/package.json

4 gradio_demo.ts

https://github.com/leoning60/browsernode/blob/ca0ba8879ebbe57667f65f48c7b34535244b4196/examples/ui/gradio_demo.ts